BACKEND ARCHITECTURE DOCUMENT

AI Developer Assistant

A FastAPI + Ollama + PostgreSQL backend for a streaming, memory-persistent developer chat assistant — built layer by layer, from scratch.

Python 3.12 | FastAPI 0.138 | Ollama (llama3.2:3b) | PostgreSQL | SQLAlchemy 2.0 | Pydantic 2

Repository
ai-developer-assistant / backend

Current phase
Phase 3 — Persistent memory

Generated
28 August 2026

Reverse-engineered from the current source tree at app/ — reflects actual code behavior, not just the README's description.

CONTENTS

00 Table of Contents

01 — INTRODUCTION

Project Overview

What this backend is, and why it exists.

AI Developer Assistant is a self-built backend for an AI coding-assistant chat product. Its explicit goal (per the project README) is educational: rather than wiring together a ready-made framework like LangChain, every layer — routing, business logic, memory, database access, and LLM integration — is implemented by hand so each design decision is understood, not just used.

The system exposes a single conversational endpoint. A client sends a message (optionally tagged with a conversation ID), and the backend streams back a token-by-token AI response — generated locally by `Ollama` running `llama3.2:3b` — while durably persisting the exchange to PostgreSQL so a conversation can be resumed later.

> **Where it stands today —** Git history shows four completed milestones: initial chat wiring, conversation-ID support, a PostgreSQL persistence layer, and a multi-prompt configuration system. There is no authentication, no tests, no containerization, and no RAG/agent tooling yet — the README frames those as future phases.

02 — TECHNOLOGY STACK

Technology Stack

Every dependency in `requirements.txt` and what role it plays.

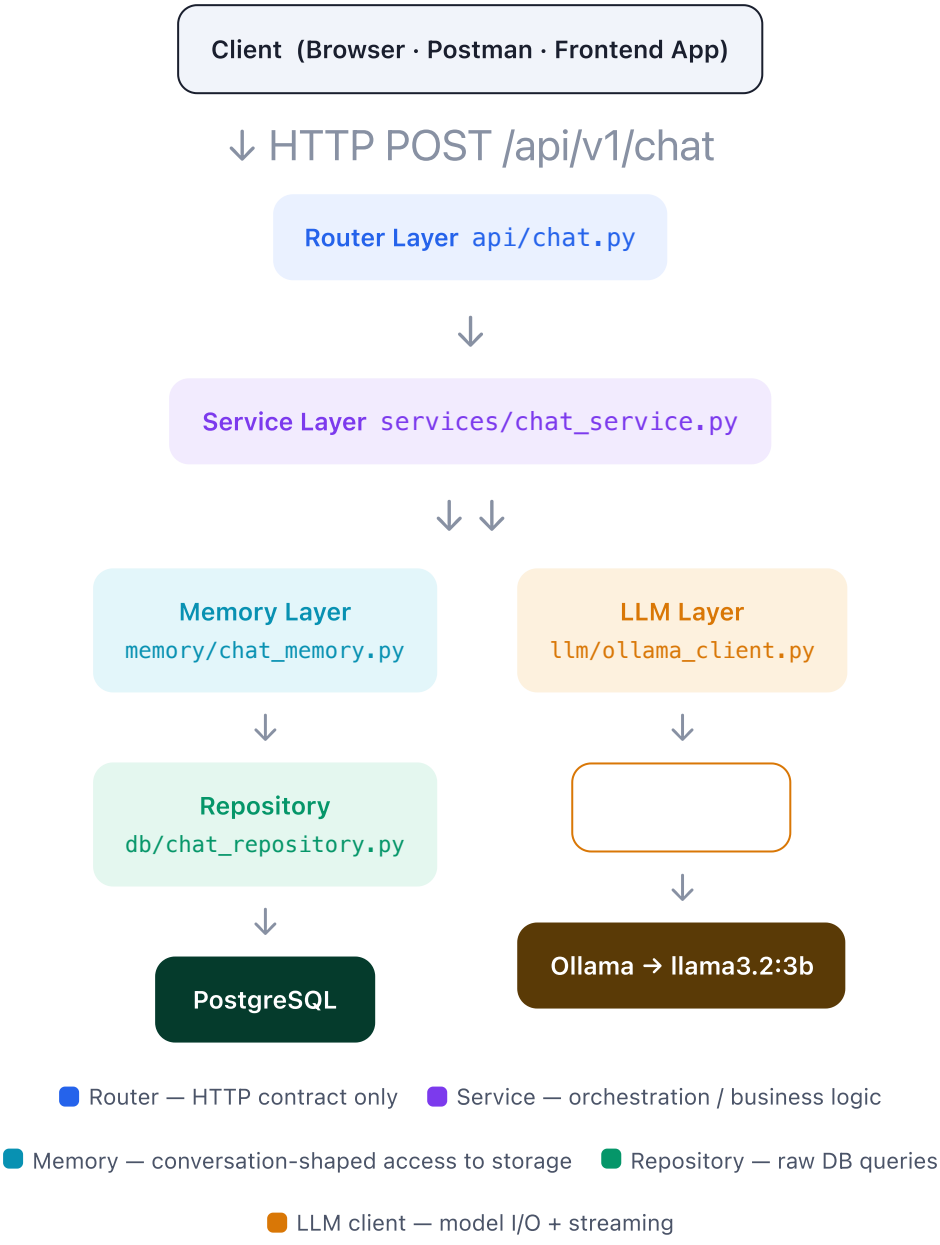
CONCERN	TECHNOLOGY	ROLE IN THIS PROJECT
Web framework	`fastapi 0.138.1`	Defines routers, request/response validation, async endpoint handling
ASGI server	`uvicorn 0.49.0`	Runs the FastAPI app (dev mode with `--reload`)
Data validation	`pydantic 2.13.4`	Request/response schemas (`schemas/chat.py`)
ORM	`SQLAlchemy 2.0.51`	Declarative models + session management for Postgres
DB driver	`psycopg2 2.9.12`	Low-level PostgreSQL connectivity used by SQLAlchemy
Database	PostgreSQL (external)	Stores conversations & messages — installed via Homebrew, not bundled
LLM runtime	Ollama (external)	Local model server — no external API cost, runs on-device
LLM client	`ollama 0.6.2`	Python client used to stream chat completions from Ollama
Model	`llama3.2:3b`	Default model; swappable in one line via `config/settings.py`

Notably absent — no `python-dotenv` / `pydantic-settings` (config is hardcoded), no `alembic` (no migrations), no test framework, no Docker tooling.

03 — SYSTEM DESIGN

High-Level Architecture

A strict layered architecture — each layer has exactly one job, and only talks to the layer directly below it.



The response path mirrors the request path in reverse, but it is **not** request-then-respond — it is a live stream. `chat_service.py` is a Python generator: as Ollama emits each token, the service `yields` it immediately through `StreamingResponse`, so the client renders text as it's generated rather than waiting for the full reply — the same UX pattern ChatGPT uses.

> **Why this shape —** because each box only knows about the one below it, any layer can be swapped without touching the others: replace Ollama with OpenAI by rewriting only `llm/ollama_client.py`; replace Postgres with Mongo by rewriting only `db/`. The service layer is the only place that knows both worlds exist.

04 — LAYOUT

Project Folder Structure

Every file that exists in the repository today, grouped by layer.

```
backend/
├── app/
│   ├── main.py                  # FastAPI app instance, router wiring, table creation
│   │
│   ├── api/
│   │   ├── chat.py               # POST /api/v1/chat — streaming endpoint
│   │   └── health.py             # GET /health — liveness check
│   │
│   ├── schemas/
│   │   └── chat.py               # ChatRequest / ChatResponse Pydantic models
│   │
│   ├── services/
│   │   └── chat_service.py       # get_ai_response_stream() — orchestration
│   │
│   ├── memory/
│   │   └── chat_memory.py        # get_history / add_message / clear_history
│   │
│   ├── db/
│   │   ├── database.py           # engine, SessionLocal, declarative Base
│   │   ├── models.py              # Conversation, Message ORM tables
│   │   └── chat_repository.py    # raw CRUD functions against the DB
│   │
│   ├── llm/
│   │   └── ollama_client.py      # Ollama client + streaming + system prompt injection
│   │
│   └── config/
│       ├── settings.py           # OLLAMA_HOST, OLLAMA_MODEL, DATABASE_URL
│       └── prompts.py            # 6 system prompt templates
│
├── requirements.txt
├── .gitignore
└── README.md
```

15 Python files total. No `tests/` directory, no `alembic/` migrations folder, no `Dockerfile`, no `.env` (though it is gitignored, implying one was planned).

05 — DEEP DIVE

Layer-by-Layer Breakdown

What each file does, exactly, and the design reasoning behind it.

ENTRY `app/main.py`

Creates the single `FastAPI()` instance (`title="AI Developer Assistant"`, `version="1.0.0"`). Immediately calls `Base.metadata.create_all(bind=engine)` — meaning the database schema is created automatically the moment the app boots, with no separate migration step. It then mounts two routers (health, chat) and defines a bare `GET /` welcome route.

> **Design note —** `create_all` only *creates missing tables*; it never alters existing ones. That's fine while the schema is this small, but it will silently stop being sufficient the moment a column needs to change — a real migration tool (Alembic) will be needed for schema evolution.

ROUTER `app/api/`

`health.py` exposes `GET /health`, returning a static `{"status": "ok", ...}` payload — useful for uptime checks or container orchestration probes later.

`chat.py` exposes `POST /api/v1/chat`. It is deliberately thin: validate the body against `ChatRequest`, then hand off directly to the service layer, wrapping its generator in a `StreamingResponse(media_type="text/plain")`. No auth, no rate limiting, no request logging happens at this layer.

SCHEMA `app/schemas/chat.py`

Two Pydantic models:

- `ChatRequest{ message: str, conversation_id: Optional[str] }` — the only input contract the API has. Omitting `conversation_id` starts a new conversation.
- `ChatResponse{ answer: str, conversation_id: str }` — defined, but **not currently used anywhere**. The live endpoint streams raw text, not this JSON shape; this schema looks like scaffolding for a future non-streaming endpoint.

SERVICE `app/services/chat_service.py`

This is the orchestrator — README calls it "the brain of the app." `get_ai_response_stream()` is a generator that, in order:

1. Generates a new `uuid4` conversation ID if none was supplied.
2. Persists the incoming user message.
3. Loads the full conversation history back out of Postgres.
4. Streams the model's reply chunk-by-chunk, `yield`-ing each piece to the caller while accumulating it into `full_response`.
5. Once the stream ends, persists the complete assistant reply as one row.

```
def get_ai_response_stream(message: str, conversation_id: str = None):
    if conversation_id is None:
        conversation_id = str(uuid.uuid4())

    add_message(conversation_id, role="user", content=message)
    history = get_history(conversation_id)
    full_response = ""

    for chunk in generate_response_stream(history):
        full_response += chunk
        yield chunk          # streamed to client immediately

    add_message(conversation_id, role="assistant", content=full_response)
```

MEMORY **app/memory/chat_memory.py**

A thin adapter between the service layer and the database repository — it exists so `chat_service.py` never has to know what a SQLAlchemy `Session` is. `get_history()` opens a session, fetches rows via the repository, reshapes them into the plain `{"role", "content"}` dict format Ollama's chat API expects, and closes the session in a `finally` block. `add_message()` similarly opens/closes a session and lazily creates the parent `Conversation` row the first time a given ID is seen. `clear_history()` is an explicit no-op, labeled as a "Phase 4" placeholder.

LLM CLIENT **app/llm/ollama_client.py**

Owns all communication with Ollama. A single module-level `Client(OLLAMA_HOST)` is shared across requests. `generate_response_stream(messages)` prepends the fixed `DEVELOPER_ASSISTANT_PROMPT` as a system message ahead of the conversation history, then calls `client.chat(..., stream=True)` and yields each non-empty content fragment as it arrives. It also times and logs (to stdout) how long the first token and the full response took — the only observability in the codebase today.

> BEHAVIOR TO KNOW ABOUT
>
> **Failures are swallowed into a friendly string, not surfaced as errors.**
>
> The whole call is wrapped in a bare `try/except Exception`. If Ollama is unreachable or the model isn't pulled, the exception is logged server-side only, and the client simply receives the text `"AI service is currently unavailable."` over an HTTP 200 — indistinguishable, from the client's point of view, from a real (if unhelpful) model answer.

DATABASE **app/db/**

`database.py` sets up the SQLAlchemy `engine` from `DATABASE_URL`, a `SessionLocal` factory, and a `DeclarativeBase` subclass (SQLAlchemy 2.0 style). No pool sizing or connection-recycling config is set — defaults are used.

`models.py` defines two tables — see the schema diagram in Section 07.

`chat_repository.py` is a flat set of functions (not a class) that take a `Session` and perform one query each: `create_conversation`, `get_conversation`, `get_All_Conversation` (currently unused — no

route lists conversations yet), `add_message`, and `get_messages`.

CONFIG **`app/config/`**

`settings.py` hardcodes three values — Ollama host, Ollama model name, and the Postgres connection string — as plain module-level constants, with a comment inviting the model to be swapped by editing one line. `prompts.py` defines six distinct system-prompt strings (general assistant, code review, debugging, concept explanation, architecture advice, code writing), each tuned with its own persona and response format. See Section 08 for how these connect (or don't) to the running system.

06 — RUNTIME BEHAVIOR

Request Lifecycle

What actually happens, in order, for a single `POST /api/v1/chat` call.

1. **Client sends the request** HTTP
   JSON body `{"message": "...", "conversation_id": null}` posted to `/api/v1/chat`.

2. **Pydantic validates the body** schemas/chat.py
   Coerced into a `ChatRequest`; a missing/wrong-typed `message` is rejected with a 422 before any business logic runs.

3. **Router opens the stream** api/chat.py
   Calls `get_ai_response_stream()` and immediately wraps it in a `StreamingResponse` — the HTTP connection stays open for the duration of generation.

4. **A conversation ID is assigned if missing** services/chat_service.py
   New conversations get a fresh `uuid4`; existing ones reuse the client-supplied ID.

5. **The user's message is written to Postgres** memory → repository → DB
   A `Conversation` row is created on first contact; a `Message` row (`role="user"`) is inserted and committed.

6. **Conversation history is read back** memory/chat_memory.py
   Up to 10 messages are fetched (see the ordering caveat in Section 09) and reshaped into `{role, content}` pairs.

7. **The system prompt + history is sent to Ollama** llm/ollama_client.py
   `DEVELOPER_ASSISTANT_PROMPT` is prepended, and `client.chat(..., stream=True)` opens a streaming completion against `llama3.2:3b`.

8. **Tokens are relayed to the client as they're generated** generator chain
   Each token flows: Ollama → `ollama_client` → `chat_service` → `StreamingResponse` → client socket, with no buffering — this is the "typing" effect.

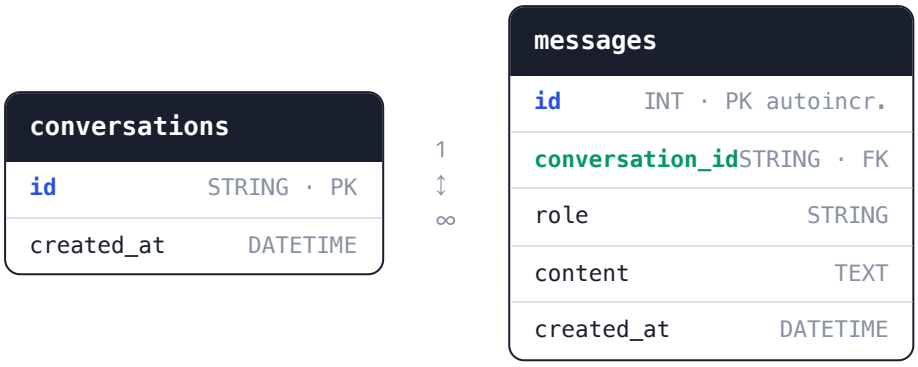
9. **The full reply is persisted once streaming ends** services/chat_service.py
   The accumulated `full_response` string is saved as one `Message` row (`role="assistant"`).

10. **Connection closes**
    The client has received plain streamed text only — never a JSON envelope, so a brand-new conversation's generated ID is not returned anywhere in this response (see Section 09).

07 — PERSISTENCE

Database Schema

Two tables, one relationship — the entirety of the current data model.



One conversation has many messages, each tagged `role="user"` or `role="assistant"` — the same shape Ollama's chat API consumes, so no transformation is needed beyond dropping SQLAlchemy's extra columns. `conversations.id` is a client-visible UUID string rather than a surrogate integer, which is what lets a client "resume" a conversation just by remembering a string.

SCHEMA GAP

No index on `messages.conversation_id`.

Unlike primary keys, PostgreSQL does not auto-index foreign key columns. Every history lookup (`WHERE conversation_id = ...`) does a full table scan once `messages` grows past a trivial size.

08 — CONFIGURATION

Configuration & Prompt System

How the assistant's behavior and connection settings are controlled.

SETTINGS Runtime configuration

`config/settings.py` is the single source of truth for three values:

CONSTANT	CURRENT VALUE	PURPOSE
`OLLAMA_HOST`	`http://localhost:11434`	Where the Ollama daemon is reachable
`OLLAMA_MODEL`	`llama3.2:3b`	Which pulled model serves chat requests
`DATABASE_URL`	`postgresql://ai_user:***@localhost/ai_assistant`	SQLAlchemy connection string

Alternative models are listed as commented-out lines directly above `OLLAMA_MODEL` (`qwen3:4b`, `qwen3:1.7b`) — swapping models today means editing source and restarting, not setting an environment variable.

PROMPTS The six system-prompt personas

`config/prompts.py` defines six distinct prompt templates, each written for a different assistant "mode":

PROMPT	PERSONA	WIRED INTO THE API?
`DEVELOPER_ASSISTANT_PROMPT`	General full-stack mentor — Python/FastAPI, React/TS, DBs, APIs, AI engineering, DevOps	**Yes — always used**
`CODE_REVIEW_PROMPT`	10+yr reviewer — bugs, security, performance, style	No route selects it
`DEBUG_PROMPT`	Root-cause-first debugger	No route selects it
`EXPLAIN_PROMPT`	Analogy-first concept teacher	No route selects it
`ARCHITECTURE_PROMPT`	Senior systems architect	No route selects it
`CODE_WRITING_PROMPT`	Clean-code generator with explanations	No route selects it

What this means today — every request, regardless of intent, is answered by the same general-purpose persona. The other five prompts are fully written and ready, but `ChatRequest` has no field (e.g. `mode`) to pick one, and `ollama_client.py` only ever imports `DEVELOPER_ASSISTANT_PROMPT`.

09 — ASSESSMENT

Known Limitations & Recommendations

Gaps between the README's "production-grade" framing and the current code — worth tracking before the next phase.

SECURITY

Database credentials are hardcoded and committed to git.

The full `postgresql://ai_user:ai_password@localhost/ai_assistant` DSN lives in `config/settings.py` as source. A `.env` is already gitignored, implying it was planned but never wired up — `python-dotenv` isn't even a dependency yet.

SECURITY

No conversation ownership or authentication.

Any client that knows (or guesses) a `conversation_id` UUID can read and append to that conversation. There's no user/session concept anywhere in the schema, so conversations aren't actually scoped to anyone.

CORRECTNESS

`get_messages(limit=10)` returns the *oldest* 10 messages, not the most recent 10.

It orders ASC by `id` then applies `LIMIT 10` — so once a conversation passes 10 messages, the model permanently loses everything after message #10 instead of sliding forward. The fix is to order DESC, limit, then reverse in Python before sending to Ollama.

API DESIGN

A freshly generated `conversation_id` is never returned to the client.

The endpoint streams raw `text/plain` only. If a client omits `conversation_id` to start a new thread, it has no way to learn the UUID the server generated — so it can't continue that same conversation on the next call.

RELIABILITY

Ollama failures are silently downgraded to a normal-looking 200 response.

The `try/except` in `ollama_client.py` yields the string `"AI service is currently unavailable."` instead of raising an HTTP error — a client (or monitoring system) can't distinguish "model said this" from "backend is broken."

PERFORMANCE

No index on `messages.conversation_id`.

Every history read does a sequential scan; fine at prototype scale, will degrade linearly as the table grows.

MAINTAINABILITY

Schema changes rely solely on `create_all` — there is no migration tool.

`create_all` never alters an existing table. The first time a column needs to be added or renamed on a table that already has data, this approach breaks down; Alembic (or similar) will be needed.

DEAD CODE

Two pieces of scaffolding aren't connected to anything yet.

`schemas/chat.py`'s `ChatResponse` and `chat_repository.py`'s `get_All_Conversation` are both defined but never referenced by any route — likely groundwork for a future non-streaming or conversation-listing endpoint.

OPS

No automated tests, no Dockerfile, no CI.

Consistent with the project's stated "learning journey" framing, but worth calling out explicitly against the README's "production-grade" language — these are the concrete gaps between the two.

10 — APPENDIX

Running the Project Locally

Condensed from the project README.

```
# 1. Environment
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# 2. PostgreSQL
brew install postgresql@16 && brew services start postgresql@16
psql postgres -c "CREATE DATABASE ai_assistant;"
psql postgres -c "CREATE USER ai_user WITH PASSWORD 'ai_password';"
psql postgres -c "GRANT ALL PRIVILEGES ON DATABASE ai_assistant TO ai_user;"

# 3. Ollama
ollama serve
ollama run llama3.2:3b

# 4. Server
python3 -m uvicorn app.main:app --reload
# → docs at http://localhost:8000/docs
```

> Tables are created automatically on first boot via `Base.metadata.create_all` — no manual migration step is required for the current schema.